

How Local Public Works, and How to Learn It in 25 Hours

Interview: Oct 8, 2026. Built from their posting, their 3 other job postings, and news coverage. Parts marked "guess" are my best reasoning, not confirmed.

1. Your three questions, answered

"Is this the WordPress of Netflix for PBS stations?"

Yes. That's almost exactly right. Their Product Manager posting says it directly: *"builds and runs streaming apps for 20+ PBS member stations. Each station gets its own branded apps on mobile and connected TV, backed by a shared CMS, API, and app store operations."*

- **CMS** (content management system) = the WordPress part. Station staff log in and pick their logo, colors, and which shows appear.
- **API** = the server the apps ask "what should I show for this station?"
- **App store operations** = publishing each station's app to every store.

Another way to think of it: **Shopify for streaming apps.** Shopify gives every shop its own branded store on one shared system. Local Public gives every station its own branded Netflix-style app on one shared system.

"Is the pay low?"

Yes. About \$55k to \$75k below the market. The typical U.S. salary for this kind of engineer in 2026 is about \$164k to \$184k, and about \$165k for remote jobs. Their pay is \$100k to \$110k, and every tech job there is in the same band (Software Engineer \$80k to \$110k, Product Manager \$100k to \$110k).

That's normal for public media. Small detail: Local Public is a "public benefit corporation." Legally that's a mission-driven company, not a nonprofit, but it's owned by Cascade PBS, which *is* a nonprofit. The perks make up for some of it: 28 days of paid time off, the week between Christmas and New Year's off, and half-day Fridays all summer.

"Why are they interviewing me?"

- **The pay keeps out most experienced specialists.** At this salary they're hiring a smart generalist who can learn, not a 10-year specialist.
- **Your experience overlaps more than you think.** You have AWS, Google Cloud, Firebase, MongoDB, Node.js, SQL, keeping production systems running (Apple), and you run your own live site.
- **Small team.** They want someone who owns problems without hand-holding. That's your Apple story.

2. Who they are (from their job postings)

- A small, fully remote team: CEO, Director of Product, a Senior Product Designer, an engineering lead, a data team, and engineers.
- **An outside development company builds much of the apps.** The Product Manager has calls with them at 8am PT. That means part of your job is likely checking and releasing code that other people wrote.

- Apps on 8 platforms: iPhone, Android, Apple TV, Amazon Fire TV, Android TV, Roku, Samsung TVs, and LG TVs. Plus a web app.
- Their careers page says the platform covers audio too, for NPR radio stations, not just video.
- Team tools: Slack, Jira (task tracking), Confluence (team wiki), Figma (designs).
- Their whole tech stack is JavaScript-based: Node.js and GraphQL.

3. The whole system on one page

YOURS = most likely your job **SHARED** = you support others **NOT YOURS** = PBS or the station handles it

A. The shows **NOT YOURS**

PBS and stations make the shows. PBS has a system called **Media Manager** that stores every video and converts it into small chunks at several quality levels. Live channels come in as a constant stream of chunks.

Tools: PBS's own systems (guess: video is served by PBS)

B. The CMS ("the WordPress part") **YOURS**

A website where station staff log in and choose logo, colors, home-screen rows, and local shows. Their choices get saved in a database.

Tools: Node.js, PostgreSQL / Aurora Serverless, MongoDB, ORM

C. The API (the menu server) **YOURS**

Every app asks: "I'm the Arizona PBS app, what do I show?" The API reads the database plus PBS's video list and answers.

Tools: Node.js + GraphQL in containers on ECS, Cloud Functions, FlightControl

D. Fast delivery near viewers **YOURS**

Copies of images, menu answers, and possibly video are kept on servers near viewers, so a Houston viewer isn't waiting on Seattle.

Tools: CloudFront (Amazon's version of this, called a CDN)

E. The apps on 8 platforms **SHARED**

One app design, many station versions. The outside developers build them. The apps report crashes and usage data.

Tools: Firebase (crash reports, analytics, settings you can change remotely)

F. Login, membership, donations **YOURS**

Members who donate unlock extra shows (PBS "Passport"). Small jobs like "is this person a member?" run as tiny on-demand functions.

Tools: Firebase, Cloud Functions (guess)

I. App store releases **SHARED**

Each station's app goes through Apple, Google, Amazon, Roku, Samsung, and LG review. Product runs it; you support it.

Tools: each store's developer console

G. Shipping changes safely **YOURS**

Code is saved to GitHub. Robots test it and deploy it. Server setup is written down as text files so it can be rebuilt with one command.

Tools: GitHub, GitHub Actions, Cloud Build, Terraform

H. Keeping it healthy **YOURS**

Dashboards, alarms, and automatic scaling so a big show doesn't crash anything. Security everywhere.

Tools: monitoring/alerting, autoscaling (ECS, Aurora Serverless, Cloud Functions)

Why both Amazon and Google? (guess) Firebase is a Google product and great for the app side (crash reports, logins). The heavier servers and database live on Amazon. Many companies end up this way by history, not by design.

4. Every tool in the posting, in plain English

Tool	What it is	Step	Closest thing you've done
AWS / GCP	Amazon's and Google's rented computers.	All	AWS at Apple. Google Cloud at Lab 916.
Node.js	JavaScript that runs on servers instead of in the browser.	B, C	You know JavaScript and PHP backends.
GraphQL	A way for an app to ask for exactly the data it needs in one request. Example: "give me station 12's name, color, and first 10 shows."	C	The REST APIs you built, but the app chooses the fields.
PostgreSQL	A SQL database, like MySQL.	B	MySQL on ranklab.org.
Aurora Serverless	Amazon's PostgreSQL that grows and shrinks its own power with traffic.	B, H	Same SQL. Amazon runs the machine.
MongoDB	A database that stores JSON-like documents instead of tables.	B	On your resume.
ORM	A code library that lets you use the database through objects instead of writing SQL by hand.	B	Your PHP data model classes at Redstone.
Container (Docker)	Your app plus everything it needs, packed in one box that runs the same anywhere.	C	Packaged Python modules at Apple.
ECS	Amazon's service that runs your boxes, keeps N copies alive, and restarts crashed ones.	C, H	SSH + scheduled jobs at Apple, automated.
FlightControl	A simpler control panel on top of ECS so deploying is easier.	C, G	None. Easy to learn.
Cloud Functions	Google's run-only-when-called code. No servers to manage.	C, F	Your Databricks serverless endpoint.
Firebase	Google's toolkit for apps: login, hosting, crash reports, remote settings.	E, F	On your resume.
CloudFront (CDN)	Copies of files kept on servers close to viewers.	D	None. Concept is simple.
GitHub Actions	Robot that runs when you push code: tests it, builds it, deploys it.	G	You use GitHub. You automated jobs at Redstone.
Cloud Build	Google's version of GitHub Actions.	G	Same idea.
Terraform (IaC)	Server setup written as text files. Run one command and it builds or changes everything to match.	G	Your automation scripts, but for servers.
Monitoring / alerting	Dashboards plus alarms that text you when something breaks.	H	Your main Apple story.

Autoscaling	Adding servers automatically when traffic jumps.	H	Capacity planning at Apple.
-------------	--	---	-----------------------------

5. Your 25-hour plan: build a mini Local Public

Why build instead of read: after this, you can truthfully say in the interview: *"To understand your platform I built a small version this week: video chunks served through CloudFront, a Node and GraphQL API on ECS, deployed by GitHub Actions, with Terraform."* That one sentence beats any amount of studying.

Before you start: in AWS, set a billing alarm at \$10 (Billing > Budgets). Delete everything in Block 12. A load balancer left running costs money every day.

Block	Hours	Do this	Step	You'll understand
1 MUST	1.5	Read this guide twice. Draw the system page from memory on paper.	All	The big picture
2 MUST	2	Install ffmpeg. Turn any MP4 into HLS chunks (an index file plus many small video files). Play it in a browser page using hls.js.	A, D	How streaming really works: chunks, not one big file
3 MUST	2	Upload the chunks to an S3 bucket. Put CloudFront in front. Play the video from the CloudFront web address.	D	CDN and caching
4 MUST	3	Build a Node.js + GraphQL API (Apollo Server) with PostgreSQL in Docker on your laptop. Two stations, each with a name, color, and show list. Query: give me station 1's shows.	B, C	The "WordPress" core: one system, many stations
5	3	Write a Dockerfile for the API. Push it to Amazon's container storage (ECR). Run it on ECS Fargate behind a load balancer.	C	Containers and ECS
6	2	GitHub Actions: on every push, build the container and deploy it to ECS.	G	Automated releases
7	2	Terraform: rebuild your S3 bucket + CloudFront from a text file. Run plan, apply, destroy.	G	Infrastructure as code
8	1.5	Firebase: host your video player page on Firebase Hosting and write one Cloud Function (for example, "is this user a member?").	E, F	The Google side
9	1.5	CloudWatch: add an alarm on API errors and an ECS autoscaling rule. Break your API on purpose and watch the alarm fire.	H	Monitoring and autoscaling
10	1	Read only: Aurora Serverless overview, how Roku and Samsung app reviews work, PBS Media Manager overview.	B, I	The rest of the vocabulary
11 MUST	3.5	Rehearse: explain the system out loud in 2 minutes, tell your 5 stories, then a mock	All	Speaking it confidently

		interview with me.		
12 MUST	1	Delete AWS resources. Push the project to your GitHub with a short README.	-	Proof you can show
	24	Plus 1 hour of buffer.		

Rule: if a block takes 50% longer than planned, stop and move on. The MUST blocks (1, 2, 3, 4, 11, 12) are about 13 hours. The rest are bonus.

Official starting pages

- hls.js demo player: hlsjs.video-dev.org/demo
- CloudFront + S3 tutorial: docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/GettingStarted.SimpleDistribution.html
- Apollo Server (GraphQL) start: apollographql.com/docs/apollo-server/getting-started
- ECS Fargate start: docs.aws.amazon.com/AmazonECS/latest/developerguide/getting-started-fargate.html
- GitHub Actions: docs.github.com/en/actions
- Terraform AWS tutorial: developer.hashicorp.com/terraform/tutorials/aws-get-started
- Firebase Cloud Functions: firebase.google.com/docs/functions/get-started
- PBS Media Manager: docs.pbs.org

6. The 2-minute system explanation (memorize by the end)

"As I understand it, you're basically Shopify for public TV streaming. PBS and the stations supply the video. You run a shared CMS where each station picks its branding and shows, and a Node and GraphQL API that every station's app calls to know what to display. Images and responses go through CloudFront so they load fast. Containers on ECS and Cloud Functions do the work, Aurora and Mongo store the data, and Firebase handles the app side like crash reports. GitHub Actions, Cloud Build and Terraform ship changes. The reliability job is keeping all of that up and fast as you grow from 20 to every station, especially during big live moments and pledge drives. Is that roughly right, and where does the video itself get served from?"

Sources: Local Public job postings (payroll.justworks.com/careers/local-public/jobs); Idealist posting; Current.org and GeekWire (July 2026); SRE salary data from recruitingfromscratch.com and jobspipe.dev; PBS Media Manager docs (docs.pbs.org).